Hardware-Aware Load Balancer for Distributed LLM Inference on CPU Systems

Aniket Raj Singh (MT25015)
Aastha Kalbhor (MT25007)
Saloni (MT25081)

April 28, 2026

Contents

Abstract

This report details our journey in building and evaluating a sophisticated load balancing system for a Large Language Model (LLM) inference service. We started by creating a foundational system with basic load balancing and then expanded it significantly in Part B. The second phase involved implementing and testing four distinct load balancing strategies: Round-Robin, Hashing, Hardware-Aware, and Least-Connection. We developed a real-time dashboard to monitor system performance and conducted rigorous benchmark tests under both normal and stress conditions. The results clearly showed that state-aware strategies, particularly Hardware-Aware and Least-Connection, offer better reliability and more stable latency under heavy load, which is a critical insight for designing robust and scalable systems. This project provides a practical look at the trade-offs between different load balancing techniques in a real-world microservices environment.

1 Problem

Distributing inference requests across multiple computational nodes in a Large Language Model (LLM) serving system presents a significant resource allocation challenge. Single-node inference systems face inherent scalability limitations, becoming bottlenecks as request volume increases. The fundamental problem is to intelligently route incoming requests to inference nodes in a manner that optimizes multiple competing objectives: minimizing request latency, maximizing system throughput, ensuring equitable resource utilization across nodes, and maintaining service availability under peak load conditions.

Naive request distribution strategies, such as round-robin routing without state awareness, fail to account for heterogeneous node characteristics and dynamic system conditions. This results in uneven load distribution, where some nodes become saturated while others remain underutilized, leading to increased tail latency, request queuing delays, and potential service degradation during high-concurrency scenarios.

2 Motivation

Effective load balancing is critical for distributed inference systems, directly impacting system performance, reliability, and resource utilization. The selection of an appropriate load balancing strategy fundamentally influences system behavior under varying operational conditions. While numerous load balancing algorithms exist—ranging from stateless approaches such as Round-Robin and Hashing to state-aware strategies like Least-Connection and Hardware-Aware routing—there is insufficient empirical analysis comparing their relative performance characteristics in LLM inference workloads.

The motivation for this study is twofold: (1) to establish a comprehensive evaluation framework for comparing distinct load balancing strategies under controlled conditions, and (2) to determine which algorithmic approach provides optimal performance characteristics with respect to reliability, latency, and throughput metrics. Understanding these trade-offs is essential for designing production-grade distributed inference systems capable of handling realistic workload patterns and stress conditions.

3 Objectives

Our main goals for this project were:

- To build a working system with multiple LLM inference servers.
- To implement and compare four different load balancing strategies.
- To create a dashboard to visualize the system's performance in real-time.
- To scientifically test our system to see which strategy works best under pressure.
- To learn about building, deploying, and managing a real-world microservices application.

4 Background

To build our system, we used a collection of modern tools and frameworks with specific versions ensuring strict reproducibility.

Backend & APIs: Our backend services and Dashboard APIs were built with **Python (3.10+)**, heavily utilizing the **FastAPI** framework, which is excellent for creating high-performance asynchronous microservices. For data validation and configuration management, we used **Pydantic** and **PyYAML**. The core LLM capability was integrated using the **llama-cpp-python** and **Ollama** libraries. To enable our real-time hardware-aware module to read raw machine-level constraints, we fundamentally relied on **psutil**.

Benchmarking & Visualization: To evaluate our load balancing mathematically and visualize off-dashboard data, we leveraged a robust scientific stack: **Matplotlib (≥3.7.0)**, **Pandas (≥2.0.0)**, **NumPy (≥1.24.0)**, and **Seaborn (≥0.12.0)**.

Frontend: We built a reactive, single-page live dashboard using **React (v18.2.0)**. We used **Vite (v5.x)** as our build tool to make development fast and efficient, and **Chart.js (v4.4.x)** via **react-chartjs-2** to map time-series data and draw the beautiful, real-time graphs on our dashboard.

Containerization: The whole system was containerized using **Docker (v24.x)**. **Docker Compose (v2.2x)** was used to manage and run all the different containers together, making our local setup and cluster lifecycle scaling a breeze.

5 Literature Review: Current LLM Load Balancing Techniques

5.1 Traditional Load Balancing vs. LLM-Specific Challenges

Traditional load balancing for web servers often expects uniform request sizes and processing times. However, Large Language Models (LLMs) present a fundamentally different workload. Generation times vary drastically based on both the input prompt complexity and the autoregressive decoding limits. This variability guarantees that blind routing logic will rapidly cause severe CPU or GPU queues.

5.2 Real-World Industry Strategies (Stateless vs. Stateful)

Stateless Strategies

1. **Round Robin / Weighted Round Robin:** Sequentially or deterministically distributes incoming traffic. While common due to its simplicity and zero-overhead nature, it typically fails in AI clusters. Since it cannot account for one node being stuck decoding an exceptionally long prompt while another node is starved of work, it frequently induces massive queuing delays and tail latency spikes during complex requests.
2. **Prompt-Aware Consistent Hashing:** This strategy generates hashes from user prompts and routes consistent paths. It operates statelessly to maintain cache-affinity (sending similar queries to the same node to reuse filesystem caching), leveraging identical queries to maximize token generation speed without strictly guaranteeing against node saturation.
3. **Randomized Routing:** Dispatches load randomly. A very low-latency balancer approach completely disjointed from hardware statuses, used primarily to avoid any central bottlenecking but completely blind to the actual state of the inference queues, resulting in arbitrary performance inconsistencies.

Stateful (LLM-Aware) Strategies

1. **KV Cache-Aware Routing:** Highly sophisticated and prevalent in GPU clusters, it tracks Key-Value cache prefixes across nodes to securely route requests to servers where prompt contexts are already maintained in memory. It drastically reduces Time to First Token (TTFT) by bypassing redundant re-computations of common context strings.
2. **Least Outstanding Tokens (LOT):** A refined generation-aware calculation that explicitly tracks the exact number of tokens each node is actively decoding. Rather than tracking "connections," it moves the workload to whichever node processes the lowest count of actively queued tokens, recognizing that every token carries a measurable compute weight.
3. **Continuous Batching & Queue State:** Real-world servers (e.g. vLLM or TGI) establish distinct prefill vs. decode queues, batching requests tightly at the inference-engine level to maximize GPU/CPU core saturation on a microsecond basis. This strategy operates at the tensor level, packing the hardware to absolute peak throughput.

5.3 Bridging Theory and Observability (Why We Created Our Strategy)

Upon deep review of real-world stateless and KV/token-stateful models, a critical pattern revealed itself: modern industry strategies treat hardware as a black box, assuming a limitless, homogeneous pool of high-end compute. They optimize aggressively for token throughput and memory states but frequently overlook the physical health and saturation of the underlying hardware itself.

We recognized that as LLM deployments increasingly expand into heterogeneous environments and edge computing arrays, relying solely on token approximations is insufficient. A node might have few outstanding tokens but still suffer from a massive CPU spike due to a complex prompt context, leading to cascading tail-latency failures.

This insight—the desire to move beyond black-box proxy metrics and actually *observe* and *react* to live hardware states—drove the invention of our **Hardware-Aware** strategy. We wanted a system that explicitly forces nodes to self-report their literal, real-time *CGroup-enforced CPU limits and RAM capabilities* via our `/stats` API. By building our own telemetry-driven metric, we created a smart failsafe that adapts on the fly to hardware starvation. Crucially, it empowered us to construct a live observability platform to visually prove, in real-time, how traffic dynamically redistributes seconds before a node would otherwise fail—a practical capability that standard token-aware routines bypass.

6 System Architecture

This document describes the high-performance, scalable system architecture for the Distributed Inference Cluster.

6.1 System Overview

The system is a horizontally scalable inference cluster designed to load balance Large Language Model (LLM) requests across an arbitrary number of containerized inference services (n nodes). It features a **Hardware-Aware** routing strategy that uses real-time node-reported telemetry to prevent saturation and optimize resource utilization across heterogeneous node configurations.

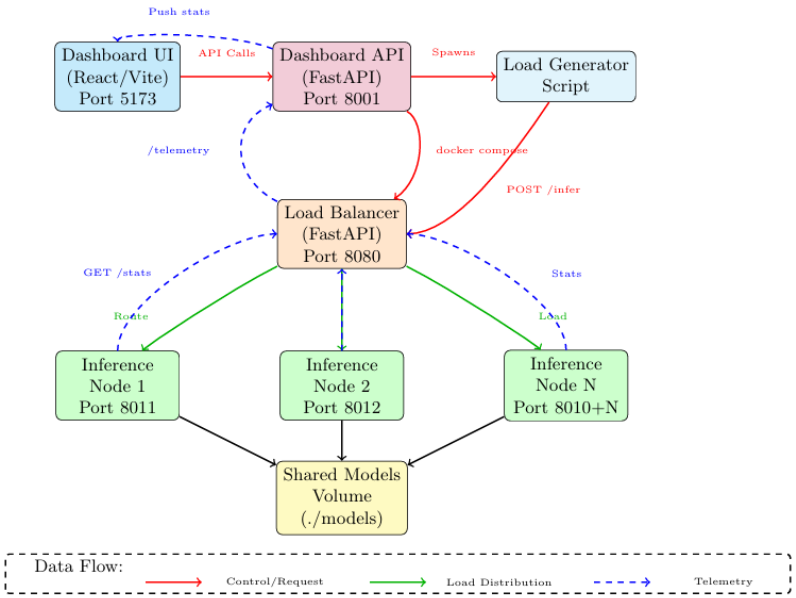

Figure 1: System architecture: Dashboard UI/API, Load Generator, Load Balancer, Inference Nodes, and Shared Models with all key connections.

Figure 1: System architecture: Dashboard UI/API, Load Generator, Load Balancer, Inference Nodes, and Shared Models with all key connections.

6.2 Key Architectural Concepts

The system is designed to be **stateless and elastic**, supporting arbitrary numbers of inference nodes:

- The `run.py` orchestrator and Dashboard API can dynamically generate a `docker-compose.yaml` configuration for any number of nodes without code modification.
- As new nodes are added to the deployment topology, the Load Balancer automatically detects them via configuration updates and begins routing requests to them immediately.
- Nodes can be added or removed without requiring a system restart or load balancer restart—only the configuration needs updating.

- This design enables transparent horizontal scaling from 1 to N nodes based on resource availability and performance requirements.

6.2.1 2. CGroup Statistics and Self-Reporting Telemetry

Each Inference Node implements **self-reporting telemetry** based on Linux container resource limits. The telemetry pipeline operates as follows:

1. **CGroup Configuration Discovery:** At startup, each node reads its own Linux CGroup files (`/sys/fs/cgroup/cpu.max`, `/sys/fs/cgroup/memory.max`, etc.) to determine its Docker-imposed CPU and memory limits.
2. **Normalized Metrics Computation:** The node computes CPU and memory utilization as a percentage of its container's limit, not the host's total resources. For example, if a container is capped at `0.5 CPU` and currently consuming `0.4 CPU`, it reports `80%` utilization instead of 8-10% (the host CPU percentage).
3. **Stats Endpoint Exposure:** Each node exposes a `GET /stats` HTTP endpoint that returns its current:
   - CPU utilization (%)
   - Memory utilization (%)
   - Active connection count
   - Request latency statistics
4. **Polling via Telemetry Collector:** The Load Balancer's embedded Telemetry Collector periodically polls every node's `/stats` endpoint (default: every 1 second) over HTTP, aggregating metrics across the entire fleet.

This design eliminates the need for centralized monitoring infrastructure while providing accurate per-node state visibility.

6.2.2 3. Hardware-Aware Burden Score

The Load Balancer employs a **weighted burden score** strategy that combines multiple telemetry signals to determine optimal request routing:

$$Score = (CPU_{utilization} \times 0.4) + (RAM_{utilization} \times 0.2) + (ActiveConnections_{normalized} \times 0.4)$$

Where:

- $CPU_{utilization}$ = CPU usage as a percentage of the container's limit (0-100%)
- $RAM_{utilization}$ = Memory usage as a percentage of the container's limit (0-100%)
- $ActiveConnections_{normalized}$ = Active connections normalized to 0-100% based on expected maximum

Routing Decision: Each incoming request is routed to the inference node with the **lowest burden score**, ensuring that:

- Nodes with high CPU utilization receive fewer requests
- Memory pressure is distributed equitably
- Active connection counts remain balanced across the fleet
- The system adapts in real-time to changing load conditions

The weighted coefficients (0.4, 0.2, 0.4) can be tuned based on observed workload characteristics and performance goals.

6.2.3 4. Docker Engine and Cluster Management

The system employs a hybrid deployment model where the **Dashboard API** (running directly on the host machine, outside Docker) manages the entire cluster lifecycle:

- **Docker Compose Orchestration:** The Dashboard API invokes `docker compose up --build` or `docker compose down` via subprocess calls on the host operating system.
- **Dynamic Configuration:** This architecture enables the Dashboard to re-deploy the entire cluster with new CPU/RAM resource configurations from the web UI without requiring manual terminal commands.
- **Subprocess Execution:** When a user requests cluster scaling or resource reconfiguration via the dashboard, the Dashboard API generates a new `docker-compose.yaml` file and executes it as a subprocess.
- **Graceful Shutdown and Restart:** Existing containers are stopped cleanly, new containers are created with updated resource limits, and the Load Balancer re-discovers the updated cluster topology.

Important Note: While the `docker.sock` socket is mounted into the Load Balancer container in some deployments, the Load Balancer's core logic does **not** use it for telemetry collection. All telemetry is gathered exclusively through HTTP polling of each node's `/stats` endpoint. The Dashboard API (running on the host) is the only component that directly manages the Docker lifecycle via `docker compose` CLI commands.

6.2.4 5. Shared Model Architecture

All inference nodes mount a single **Shared Models Volume** from the host filesystem (`./models`). This architecture provides several benefits:

- **Storage Efficiency:** The quantized GGUF LLM model file (typically 600-700 MB) is stored only once on the host disk, not replicated across each container.
- **Reduced I/O:** Model loading is significantly faster after the first node loads it, as subsequent nodes benefit from OS filesystem caching.
- **Cost Effectiveness:** In production deployments, this shared volume approach reduces overall storage requirements linearly with the number of nodes.
- **Model Updates:** Updating the LLM model requires updating only a single file; all nodes automatically load the updated model on next access or restart.

The shared volume is mounted at `/models` inside each container and at `./models` on the host, enabling all inference nodes to reference identical model files.

6.3 Core Components

1. Load Balancer (Routing Layer): The load balancer is the central traffic orchestrator, deployed as a single FastAPI application on port 8080. Its key responsibilities include:

- **Dynamic Node Discovery:** Reads the configured list of inference nodes from `group22_config.py` or environment variables, enabling flexible cluster management without code changes
- **Request Routing:** Routes incoming inference requests to appropriate backend nodes based on the active strategy (Round-Robin, Hashing, Hardware-Aware, or Least-Connection)
- **Telemetry Collection:** Continuously polls all configured inference nodes every 2 seconds via their `/stats` endpoints to gather real-time metrics: CPU utilization, memory usage, and active connection counts
- **Health Monitoring:** Tracks node health status and avoids routing requests to unhealthy nodes, enabling graceful degradation during node failures
- **Dashboard API:** Exposes the `/dashboard-api` endpoint that serves aggregated system metrics and per-node statistics to the frontend dashboard

2. Inference Nodes (Inference Layer): Each inference node is an independent FastAPI service running the LLM via llama-cpp-python. The system is designed with **dynamic node discovery**, allowing nodes to be added, removed, or modified without restarting the load balancer. Nodes are configured through one of two mechanisms:

- **Configuration File:** `group22_config.py` defines the list of inference nodes that the load balancer will discover
- **Environment Variables:** The `NODES` environment variable can override the configuration, allowing runtime node specification via `node1:8000,node2:8000,node3:8000` format

The default deployment demonstrates the system with three heterogeneous nodes (serving as a test case for evaluating performance under heterogeneous resources), but the architecture supports scaling to any number of nodes. Each node exposes:

- `POST /inference` - Accepts inference requests and returns LLM-generated responses
- `GET /stats` - Returns current CPU usage, memory consumption, and active connection count

The heterogeneous resource allocation in test deployments (e.g., 1.0 CPU, 0.5 CPU, 1.0 CPU) is intentional, designed to evaluate how different load balancing strategies adapt to nodes with different computational capacity.

3. Real-Time Dashboard & Observability (Monitoring/Client Layer): A React-based single-page application (SPA) that provides deep visual insights into system behavior, going beyond simple logs by proving our Hardware-Aware strategy works in real-time. It operates on a Reactive architecture where the Vite frontend maintains a persistent polling loop with the Load Balancer's metrics. Note specifically:

- **Data Transformation & Time-Series Rendering:** Raw backend telemetry is instantly parsed and converted into continuous time-series arrays for rich Chart.js rendering.

- **Live Percentile Tracking:** Continuously calculates and visualizes p50, p95, and p99 latency percentiles to expose hidden tail-latency spikes that simple averages would dangerously mask.
- **Hardware Saturation Mapping:** Displays active CPU and RAM utilization per container, mapped exactly against the configured Docker CGroup limits rather than global host metrics.
- **Dynamic Traffic Re-routing Proof:** Under synthetic stress tests, visually captures the exact moment a node hits a compute bottleneck (e.g., exceeding 90% CPU) and immediately graphs the incoming traffic migrating to underutilized nodes.
- **Instant Strategy Comparison:** Allows seamless, side-by-side behavioral comparison of stateless (e.g., Round-Robin) and stateful (e.g., Hardware-Aware) routing under identical synthetic load patterns.
- **Success Rate & Graceful Degradation Monitoring:** Tracks live request success and failure rates across the entire cluster, immediately flagging degraded inference nodes.
- **Asynchronous Data Polling Architecture:** Utilizes an optimized React `useEffect` polling loop communicating directly with the FastAPI `/dashboard-api` every 1-2 seconds, ensuring zero frontend render-blocking.
- **Cluster Lifecycle Feedback:** Visually reflects real-time topology changes when new inference nodes are dynamically spawned, scaled out, or destroyed by the backend orchestrator.

The dashboard polls the load balancer's `/dashboard-api` endpoint every 1-2 seconds to fetch fresh metrics, updating interactive visualizations in real-time.

4. Telemetry Collector (Monitoring Layer): A background telemetry service running within the load balancer that:

- Asynchronously polls all inference nodes at configurable intervals (default: 2 seconds)
- Aggregates metrics and maintains time-series history for trend analysis
- Implements health checks and automatic failure detection
- Serves collected data to both the routing logic and the dashboard API

6.4 Data Flow Choreography

6.4.1 Request Processing Flow

1. **User Interaction:** The user interacts with the **Dashboard UI** (React/Vite application on port 5173) to trigger load generation or system monitoring.
2. **Dashboard API Invocation:** The Dashboard UI sends API calls to the **Dashboard API** (FastAPI on port 8001).
3. **Load Generator Spawning:** The Dashboard API spawns a subprocess executing the **Load Generator Script**, which begins generating synthetic inference requests.
4. **Load Balancer Routing:** The Load Generator submits requests to the **Load Balancer** (`POST /infer` on port 8080).

5. **Intelligent Request Distribution:** The Load Balancer computes the burden score for each node and routes the request to the node with the lowest score.

6. **Node Processing:** The selected inference node processes the request using its local LLM model (`/models/model.gguf`) and returns the result.

7. **Response Path:** The Load Balancer receives the response and forwards it to the Load Generator and Dashboard API.

6.4.2 Telemetry Collection Flow

1. **Periodic Polling:** Every 1 second (configurable), the Load Balancer's Telemetry Collector issues `GET /stats` requests to all inference nodes.

2. **Self-Reported Metrics:** Each node returns its current CPU utilization, memory utilization, active connection count, and latency statistics.

3. **Aggregation:** The Telemetry Collector aggregates these metrics and stores them in memory, maintaining a time-series history for trend analysis.

4. **Dashboard API Query:** The Dashboard API periodically polls the Load Balancer's `GET /telemetry` endpoint to retrieve aggregated metrics.

5. **Dashboard Visualization:** The Dashboard API pushes these metrics to the Dashboard UI, which displays real-time charts and performance graphs using Chart.js.

6. **Verification and Analysis:** Users can verify system performance, identify bottlenecks, and make informed decisions about scaling or strategy adjustments.

6.5 Component Responsibilities

Component	Location	Key Responsibilities
Dashboard UI	Host (Port 5173)	User interactions, visualization, load test configuration
Dashboard API	Host (Port 8001)	Cluster lifecycle mgmt, Docker Compose orchestration
Load Generator	Host	Generates synthetic load based on dashboard instructions
Load Balancer	Docker (Port 8080)	Request routing, telemetry collection, strategy execution
Inference Nodes	Docker (Ports 8011-801N)	LLM inference, self-reported telemetry via /stats
Shared Volume	Host (./models)	Centralized storage for GGUF LLM model file

Table 1: Component responsibilities in the distributed inference architecture.

6.6 Deployment Configuration

6.6.1 Default Docker Compose Topology

Service	Container Name	Port	Resource Limits
Load Balancer	`load_balancer`	8080	Configurable
Inference Node 1	`inference-node-1`	8011	1.0 CPU, 512 MB RAM
Inference Node 2	`inference-node-2`	8012	0.5 CPU, 512 MB RAM
Inference Node N	`inference-node-N`	8010+N	Configurable

Table 2: Default Docker Compose deployment showing heterogeneous node configurations for testing load balancing under resource constraints.

6.7 Scaling and Extensibility

6.7.1 Adding New Inference Nodes

New nodes can be added by either:

1. **Configuration File Method:** Modify the node list in the `docker-compose.yaml` or configuration file and restart the cluster.
2. **Runtime Discovery Method:** Update the `NODES` environment variable to include the new node endpoints, and the Load Balancer will auto-discover them on next telemetry poll.

The Load Balancer automatically begins routing to new nodes without requiring a restart, enabling true elastic scaling.

6.7.2 Implementing New Routing Strategies

New load balancing strategies can be added to the system by:

1. Implementing a new strategy function in the `group22_middleware.py` file.
2. Registering the strategy with a unique name in the strategy dispatcher.
3. Updating the configuration parameter to activate the new strategy.
4. Restarting the Load Balancer service to enable the new strategy.

The Telemetry Collector remains unchanged, as all strategies operate on the same aggregated metrics.

6.7.3 Extending Telemetry Metrics

The telemetry system is designed for extensibility:

1. Add new metrics to the inference node's `GET /stats` endpoint response.
2. Update the Telemetry Collector logic to parse and aggregate the new metrics.

3. Modify the burden score formula to incorporate new metrics if desired.
4. Add new visualizations to the Dashboard UI to display the additional metrics.

6.8 System API Specifications

In proving the complexity of the microservices ecosystem, we strictly document six primary APIs facilitating cluster communication:

- `POST /infer` **(Load Balancer):** The public-facing entry point. Receives synthetic load and dynamically calculates the next node jump.
- `GET /telemetry` **(Load Balancer):** Asynchronously aggregates and computes the internal raw states of all connected containers.
- `GET /dashboard-api` **(Load Balancer):** Formats raw latency blocks and hardware utilization logic specifically arrayed for React/Chart.js rendering.
- `POST /inference` **(Inference Node):** Internal cluster node execution point that triggers the `llama-cpp-python` pipeline for text generation.
- `GET /stats` **(Inference Node):** The core architectural pivot of the system; nodes broadcast normalized current `CGroup` saturation (CPU, RAM) preventing immediate orchestration crashes.
- **Cluster Lifecycle Orchestrator (Dashboard Backend - Port 8001):** Controls Docker Compose, dynamically creating, spinning up, destroying, scaling, and managing the overall `yaml` states without OS bash administration.

7 Methodology/Design

Our system is designed as a microservices architecture. This means that instead of one big application, we have several smaller, independent services that talk to each other.

- **Inference Service:** This is the worker bee. We can run multiple instances of this service, and each one runs the LLM to process user requests. It also has a special `/stats` endpoint that reports its current CPU usage, memory, and active connections.
- **Load Balancer:** This is the traffic cop. It sits in front of all the inference services and decides where each incoming request should go. This is where we implemented our four strategies:
    1. **Round-Robin:** The simplest strategy. It just cycles through the list of servers.
    2. **Hashing:** It uses the request's content to generate a hash, which determines which server gets the request. This ensures the same request always goes to the same server.
    3. **Hardware-Aware:** A smarter strategy. It periodically asks each server for its hardware stats and sends the request to the server with the most available resources.
    4. **Least-Connection:** Another intelligent strategy. It keeps track of how many active connections each server has and sends the new request to the one that is currently the least busy.
- **Dashboard:** This is our window into the system. It's a React web application that communicates with the load balancer to get the latest telemetry data and displays it on a series of live charts.

8 Implementation

The implementation was a journey. We started by creating the Dockerfile for our inference service, making sure it could load the LLM and serve requests. Then, we built the load balancer using FastAPI's middleware feature, which allowed us to intercept incoming requests and apply our routing logic.

The core of the implementation was in the `group22_middleware.py` file, where we wrote the Python code for each of the four balancing strategies. We used the `httpx` library to gather telemetry data from the inference nodes for the hardware-aware strategy.

For the dashboard, we set up a React project using Vite. The main component, `App.jsx`, fetches data from our load balancer's `/dashboard-api` endpoint every few seconds and updates the charts, creating the live monitoring experience.

Finally, we wrote a `run.py` script to automate the entire process of starting the system locally.

9 Evaluation & observations

After building the system, it was time to test it. We used a benchmarking tool, implemented in `group22_load_generator.py`, to simulate both a normal load and a stress test scenario for each of the four strategies. The results, shown in the figures below, clearly highlight the differences between stateless and state-aware routing approaches.

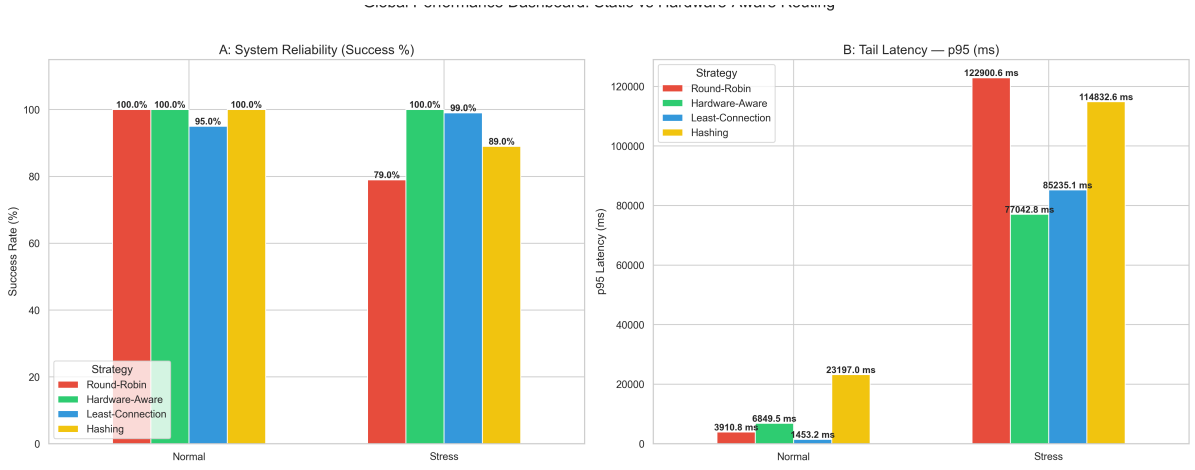

Figure 2: Global Performance: System Reliability and Tail Latency under Normal vs. Stress loads.

The updated results in Figure 2 show a much clearer difference between stateless and state-aware routing strategies. Under normal load, Round-Robin, Hardware-Aware, and Hashing all achieved a perfect 100% success rate, while Least-Connection achieved 95%. In terms of tail latency, Least-Connection performed the best during normal load with a p95 latency of only 1453.2 ms, followed by Round-Robin at 3910.8 ms, Hardware-Aware at 6849.5 ms, and Hashing performing the worst at 23197.0 ms.

Under stress conditions, the differences became much more obvious. Hardware-Aware routing achieved the best reliability, maintaining a perfect 100% success rate even under heavy load.

Least-Connection was very close behind with 99% success, while Hashing dropped to 89% and Round-Robin performed the worst at only 79%.

The latency values under stress also show the benefit of state-aware routing. Hardware-Aware had the lowest p95 latency at 77042.8 ms, making it the best overall strategy under heavy load. Least-Connection had the second-best p95 latency at 85235.1 ms. In comparison, Hashing and Round-Robin suffered from very large latency spikes, reaching 114832.6 ms and 122900.6 ms respectively. This shows that state-aware approaches can react better to overloaded nodes, distribute requests more evenly, and avoid extreme queuing delays.

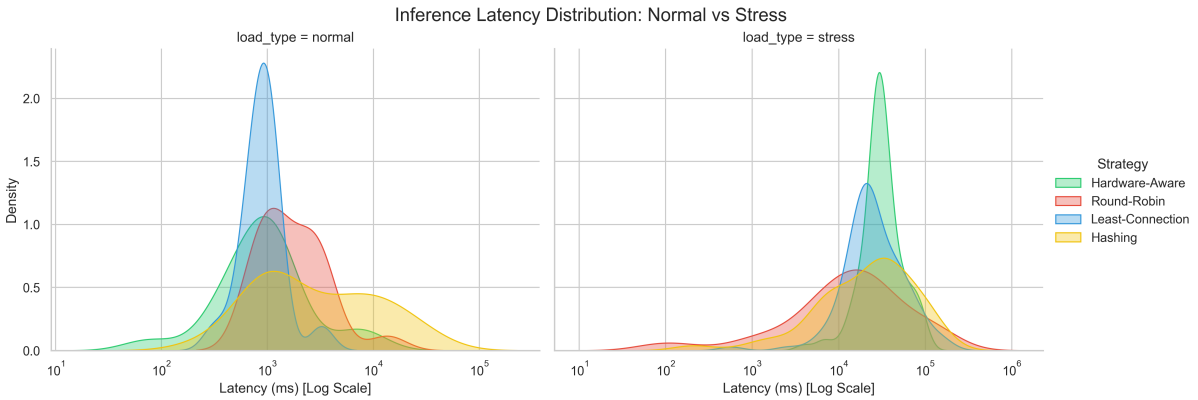

Figure 3: Inference Latency Distribution for all strategies under Normal vs. Stress loads.

The latency distribution curves in Figure 3 provide additional insight into how each routing strategy behaves. Under normal load, Least-Connection shows the most concentrated latency distribution around lower values, meaning requests are completed more consistently and with fewer spikes. Hardware-Aware and Round-Robin show slightly wider distributions, while Hashing has the widest spread and the heaviest tail, indicating more variability and occasional very slow requests even during normal operation.

Under stress conditions, all strategies shift toward much higher latency ranges, but the shape of the curves is important. Hardware-Aware maintains the tightest and most concentrated distribution, meaning most requests still finish within a relatively controlled range. Least-Connection also performs well, although its distribution becomes slightly wider under heavy load. Round-Robin and Hashing show much broader distributions with heavier tails extending to very large latency values, indicating that many requests spend a long time waiting in overloaded queues.

Overall, the updated latency distributions confirm that Hardware-Aware and Least-Connection are more robust under stress because they adapt to current node conditions, whereas Round-Robin and Hashing continue routing blindly without considering server load.

10 Outcomes & Conclusion

This project was a very useful hands-on experience in designing and building a distributed system. One important takeaway is that state-aware load balancing becomes much more valuable when the system is pushed under stress. Simple stateless strategies like Round-Robin and Hashing are easy to implement, but they are less reliable under heavy load because they do not consider the actual condition of the servers before routing requests.

From our updated results, Hardware-Aware routing emerged as the strongest overall strategy under stress, since it achieved both the highest success rate and the lowest p95 latency. Least-Connection also performed very well and remained highly reliable, especially because it was able to keep latency relatively controlled compared to the stateless methods.

Overall, the data shows that using real-time node information leads to better routing decisions, lower tail latency, and improved system stability. Building the live dashboard also helped us understand the system more clearly, because it turned raw numbers into an easy visual comparison of how each strategy behaves under normal and stress conditions.

11 References

- **FastAPI**: A modern, fast (high-performance), web framework for building APIs with Python 3.7+ based on standard Python type hints. `https://fastapi.tiangolo.com/`
- **Docker**: An open platform for developing, shipping, and running applications. `https://www.docker.com/`
- **React**: A JavaScript library for building user interfaces. `https://reactjs.org/`
- **Vite**: A build tool that aims to provide a faster and leaner development experience for modern web projects. `https://vitejs.dev/`
- **Chart.js**: A simple yet flexible JavaScript charting library for designers & developers. `https://www.chartjs.org/`
- **llama-cpp-python**: Python bindings for the llama.cpp library. `https://github.com/abetlen/llama-cpp-python`